

SonicBoom: 第三次アウトオブオーダプロセス

**Jerry Zhao, Ben Korpan, Abraham Gonzalez,
Krste Asanovic**

英語論文購読第2回 ヴハイナム

Table of Content

1. Introduction
2. BOOM history
3. Instruction Fetch
4. Execute
5. Load-Store Unit and Data Cache
6. System support
7. Evaluation
8. What's next
9. Conclusion

Introduction

- Deployment of high-performance superscalar, out-of-order (OOO) cores expanding into mobile and edge devices
- Security, power, performance, area need to be considered when evaluating new design
- An open-source hardware implementation of a superscalar, OOO core is invaluable

- Open source hardware implementation have numerous advantages over software models of high-performance cores
 - Demonstrate precise microarchitectural behaviors
 - Execute real applications for trillions of cycles
 - *Empirically* provide power and area measurements
 - Point of comparison for new microarchitectural platform

- Most of the current open-source hardware development framework only support simple in-order cores
 - Rocket
 - Ariane
 - Black Parrot
 - PicoRV32
- Modern mobile / server-class SOC is impossible without a full-featured, high performance implementation of a superscalar OOO core.

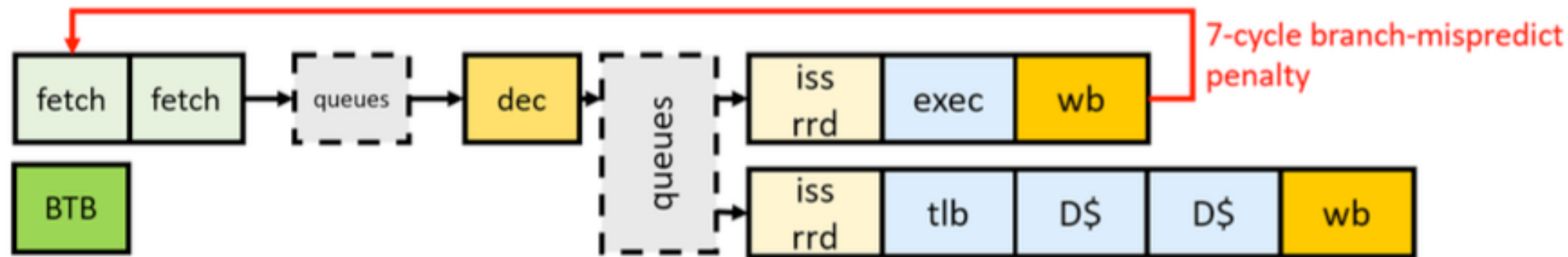
Fastest open-source core

Table 1: Comparison of open-source out-of-order cores.

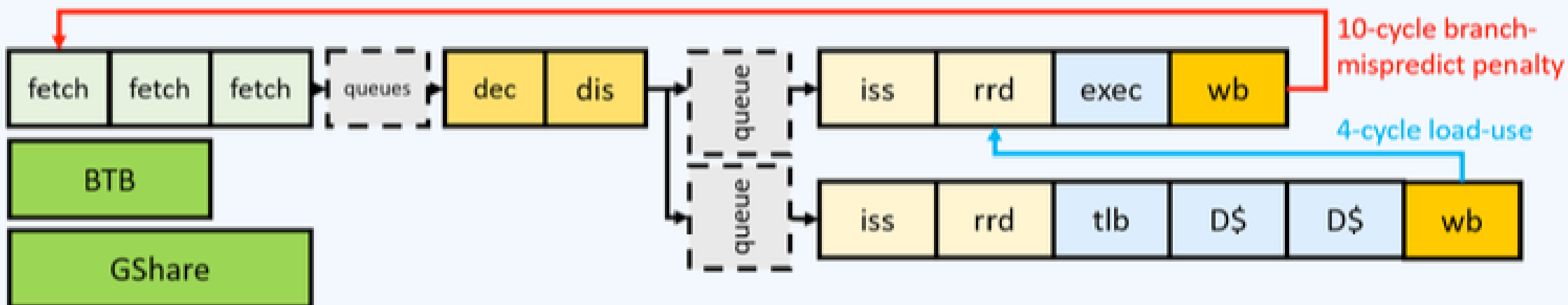
	<i>Sonic BOOM</i>	BOOMv2	riscy- OOO	RSD
ISA	RV64GC	RV64G	RV64G	RV32IM
DMIPS /MHz	3.93	1.91	?	2.04
SPEC06 IPC	0.86	0.42	0.48	N/A
Branch Pre- dictor	TAGE	GShare	Tourney	GShare
Dec Width	1-5	1-4	2	2
Mem Width	16b/cycle	8b/cycle	8b/cycle	4b/cycle
HDL	Chisel3	Chisel2	BSV+CMD	System Verilog

BOOM history

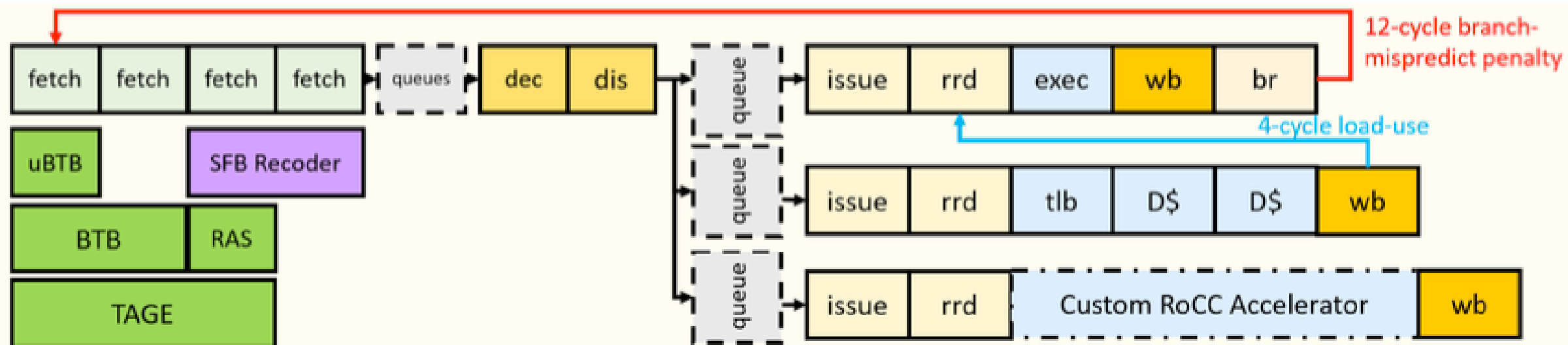
- BOOM Version 1
 - Education tool for University
 - Based on microprocessor MIPS R10K
(A RISC implementation of MIPS IV ISA)
 - Not enough pipeline stages, thus
not physically realisable



- BOOM Version 2
 - Add more pipeline to Front-end and Execution paths
 - Split floating-point register file and execution units to independent pipeline
 - Split issue queues to separate queues:
Int, Memory, Float operations



- Sonic BOOM
 - Support more software stacks
 - Fix performance bottlenecks, keep physical realizability
 - IF
 - EX backend
 - Load / Store
 - Improved critical path delay



- In short
 - If BOOMv1 was created for educational purpose
BOOMv3 (SonicBOOM) was created for research in
hi-performance core design

Instruction Fetch

- Support for 2-byte RVC instructions
- Implementation of TAGE branch predictor
- Multiple branch-resolution units

- Support for 2-byte forms of common 4-byte instructions
 - Default RV-ISA subset for packaged Linux distributions (Fedora, Debian)
 - New fetch-unit which decodes possible 2 / 4 bytes instruction for every 2 bytes

- Implemented branch predictors
 - Get banked to match to ICache
 - Using TAGE and RAS to improve the predictor accuracy, reduce miss after returning from a function
- Using multi-branch resolution unit to evaluate multiple B-Instruction per cycle

Execute

- Support for RoCC (Rocket Custom Coprocessor) helps integration of custom processor to BOOM pipeline
- Optimized the implementation of SFB (Short Forward Branch) by recoding difficult-to-predict branches to predicated microOps

- Short Forward Branch:
 - Data-dependent branch over short basic block is common. This lead to high chances of mispredictions and pipeline flushes
 - Short branch get converted to "set-flag" and "conditional-execute" micro-ops (e.g. set.ge micro-op)

C	Assembly	Executed MicroOps
<pre> int max = 0; int maxid = -1; for (i = 0; i < n; i++) { if (x[i] >= max) { max = x[i]; maxid = i; } } </pre>	<pre> loop: lw x2, 0(a0) bge x1, x2, skip mv x1, x2 mv a1, t0 skip: addi a0, a0, 4 addi t0, t0, 0x1 j loop: </pre>	<pre> loop: lw x2, 0(a0) set.ge x1, x2 p.mv x1, x2 p.mv a1, t0 addi a0, a0, 4 addi t0, t0, 0x1 j loop: </pre>

Branching can be omitted with the use of micro-op
This led to **1.7 times more** Instruction per Cycle

Load-Store Unit, Data Cache

L1 Cache drawbacks:

- 1-width interface is a bottleneck of BOOM's fetch and decode pipeline
- Non-speculative caching system
- Blocking load refills on cache eviction
This leads to increased cycles in evicting the replaced line

- Dual-ported L1 Data Cache
 - Data cache separated into 2 banks
Each bank is a 1R1W SRAM
 - Load-store unit is upgraded accordingly
to support dual-ported Data cache

- Improved L1 Performance
 - Cache-eviction can be done in parallel with cache refill. Refill data is written into line-fill buffer
When Cache-eviction completed, line-fill buffer get flushed into cache arrays
 - Next-line prefetcher between L1 and L2
Speculatively fetches sequential cache lines after MISS into line buffers
 - Missspeculated cache refills can be flushed out of L1. Only write to L1 cache if the request is correctly speculated

